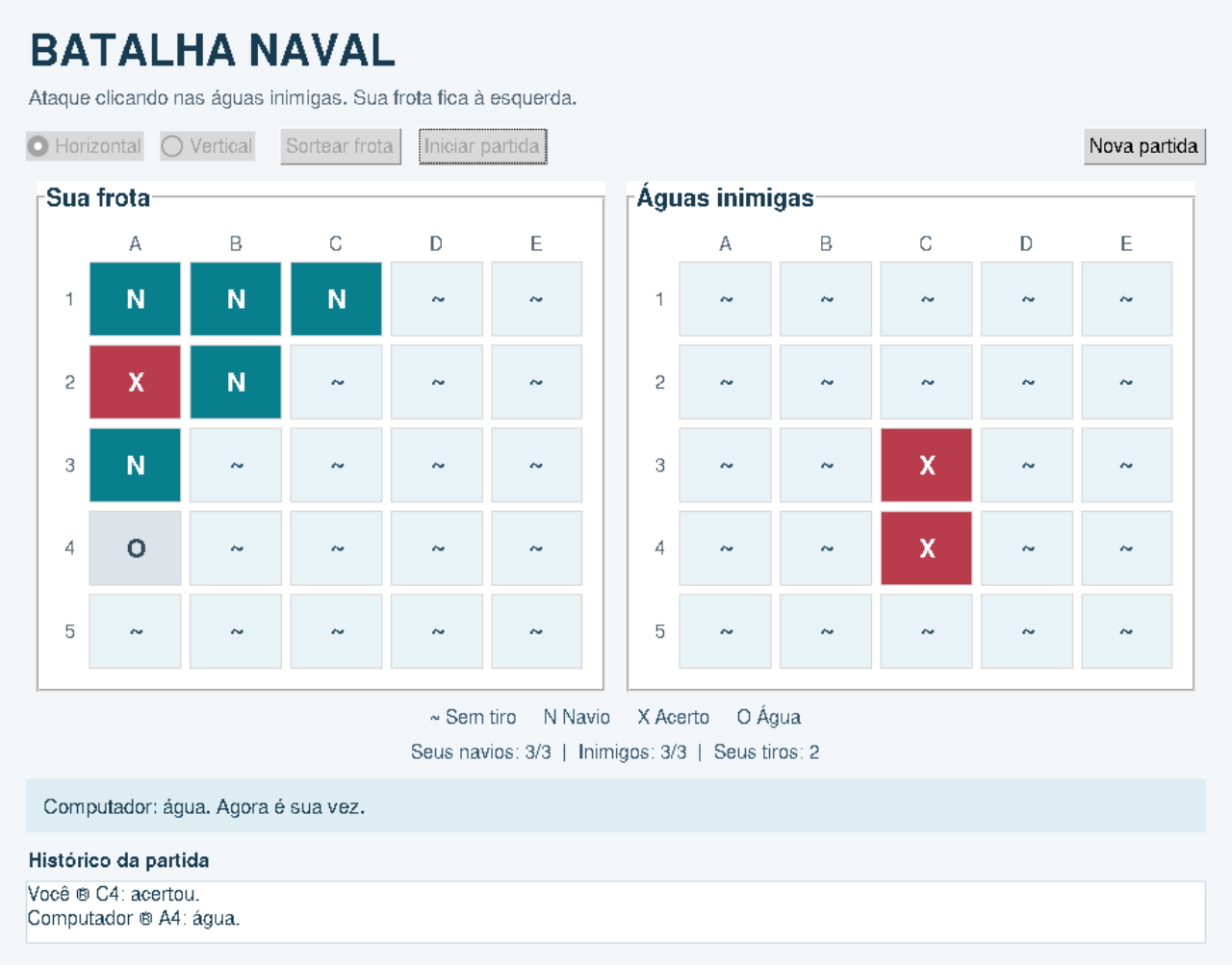
PYTHON • ORIENTAÇÃO A OBJETOS • TKINTER

Manual de construção Batalha Naval com interface gráfica

Versão 2 — aprenda fazendo, etapa por etapa.

Requisitos do trabalho, blocos para construir os arquivos, explicação das **423 linhas**, checkpoints e testes com a janela real.



Captura real da versão entregue, após dois ataques e duas respostas do computador.

O jogo usa a biblioteca padrão do Python. Este PDF e os testes são materiais de apoio; o programa completo acompanha o pacote.

Roteiro do manual

1. Como estudar e o que você vai construir **4**

1.1. O mínimo de Python para acompanhar 4

1.2. Coordenadas e responsabilidades 5

2. Preparar o computador e experimentar a versão pronta **5**

Passo 1 — Instale Python e confira o Tkinter 5

2.1. Abra a nova versão 6

2.2. Prepare a frota por cliques 6

2.3. Ataque e acompanhe os turnos 7

3. Construir as regras: batalha_naval.py **9**

Passo 2 — Modele o navio 9

Passo 3 — Crie o tabuleiro e valide coordenadas 10

Passo 4 — Implemente o posicionamento completo 11

Passo 5 — Processe os tiros e o afundamento 13

Passo 6 — Faça a exibição textual 14

Passo 7 — Implemente o jogador e a derrota 15

Passo 8 — Converta coordenadas e sorteie a frota 16

Passo 9 — Complete a versão de terminal e a demonstração 18

4. Construir a janela: batalha_naval_gui.py **23**

4.1. Entenda a programação por eventos 23

Passo 10 — Importe as regras e defina as cores 23

Passo 11 — Defina a classe da janela e suas variáveis 25

Passo 12 — Monte a estrutura da interface 26

Passo 13 — Crie as grades e associe os cliques 28

Passo 14 — Escreva o histórico 29

Passo 15 — Prepare o cancelamento de respostas 30

Passo 16 — Inicialize uma nova partida 30

Passo 17 — Posicione um navio pelo clique 31

Passo 18 — Ofereça o sorteio da própria frota 32

Passo 19 — Libere o início apenas quando a frota estiver pronta 33

Passo 20 — Ataque por clique e agende a resposta 34

Passo 21 — Execute a resposta do computador 35

Passo 22 — Centralize o encerramento 36

Passo 23 — Sincronize símbolos, controles e contadores 36

Passo 24 — Feche sem deixar callbacks pendentes 39

Passo 25 — Abra a janela e inicie o laço de eventos 39

5. Testar de verdade: regras, widgets e eventos 41

Passo 26 — Execute a suíte e interprete o resultado 41

5.1. Como escrever um teste pequeno 41

5.2. Como a janela foi validada neste ambiente 42

6. Conferência final e dúvidas comuns 43

Passo 27 — Faça a conferência da entrega 43

6.1. Exercícios após concluir o manual 44

6.2. Arquivos do pacote 45

1. Como estudar e o que você vai construir

Este manual ensina a fazer o projeto em ordem: primeiro as regras, depois a janela e, por fim, os testes. Em cada passo você verá o objetivo, o arquivo que deve editar, o código a acrescentar, a explicação de cada linha e uma verificação para saber se pode continuar.

Para reconstruir, crie uma pasta separada chamada batalha_naval_estudo. Mantenha o pacote pronto em outra pasta para consulta. Digite os blocos ou copie os trechos da versão textual manual_v2.md. Não substitua partes prontas sem conferir a ordem dos passos.

Os códigos impressos mantêm a indentação original. Linhas longas podem ser quebradas visualmente no PDF para caber na página; isso não cria uma linha nova no arquivo. Os arquivos .py e a versão textual contêm as linhas completas. O número Linha N sempre se refere ao arquivo indicado no início do passo.

Há duas numerações independentes: batalha_naval.py tem 200 linhas; batalha_naval_gui.py tem 223. A explicação inclui linhas em branco. Ferramentas de geração de PDF e scripts de teste são apoio e não participam de uma partida.

O que é obrigatório no PDF do trabalho	Onde é resolvido
Navio.esta_afundado()	Conta partes atingidas e compara com o tamanho.
Tabuleiro.posicionar_navio()	Verifica todas as casas antes de confirmar: limites, sobreposição e cruzamento.
Tabuleiro.receber_tiro()	Identifica água/acerto/afundamento, altera o estado e aplica dano.
Jogador.atacar()	Chama o tabuleiro adversário e registra o resultado no próprio histórico.
Jogador.verificar_derrota()	Verifica a frota inteira; uma frota vazia não conta como derrota.

A janela, o tabuleiro 5×5, a frota 3/2/1 e os botões são escolhas adicionais para tornar a entrega utilizável. Navios adjacentes são permitidos; compartilhar uma casa é proibido. A interface gráfica reutiliza as regras: um clique apenas chama os métodos que já foram testados.

1.1. O mínimo de Python para acompanhar

Variável é um nome associado a um valor ou objeto. **Lista** guarda elementos em ordem e pode mudar. **Tupla**, como (1, 2), representa um par fixo. **Dicionário** associa uma chave a um valor. A grade usa listas; o histórico usa coordenadas como chaves de um dicionário.

Classe define um tipo de objeto. Navio(3) cria uma instância. **Atributos** guardam seu estado, e **métodos** são funções da classe. self identifica a instância que recebeu a chamada. __init__ prepara seus atributos. Não passe self manualmente ao chamar navio.esta_afundado().

def define uma função; **if/elif/else** escolhem caminhos; **for** percorre uma sequência; **while** repete enquanto uma condição permitir. return encerra a função; break encerra o laço; continue passa à próxima iteração. Os dois-pontos e a indentação definem os blocos.

= atribui; **==** compara valores; **is** compara a identidade do objeto. None representa ausência. True e False são valores booleanos. raise ValueError sinaliza um valor inválido; try/except permite apresentar esse erro ao usuário sem quebrar a interação.

Uma compreensão como [(l, c) for l in range(5) for c in range(5)] equivale a dois laços e um append, criando 25 pares. range(5) gera 0, 1, 2, 3 e 4. Uma f-string, como f"Tiro: {resultado}", coloca o valor da variável dentro do texto.

1.2. Coordenadas e responsabilidades

Na tela	No código (linha, coluna)
A1	(0, 0)
B2	(1, 1)
D3	(2, 3)
E5	(4, 4)

Jogador.atacar chama Tabuleiro.receber_tiro. O tabuleiro localiza a referência ao Navio, registra o dano e consulta esta_afundado. O resultado volta ao jogador e entra no seu registro. A janela mostra esse resultado e escolhe a próxima fase. Cada parte tem uma responsabilidade clara.

2. Preparar o computador e experimentar a versão pronta

Passo 1 — Instale Python e confira o Tkinter

Instale Python 3 com suporte a Tcl/Tk. O jogo não exige pygame, navegador, banco de dados ou pacotes de pip. Tkinter integra a biblioteca padrão, mas algumas distribuições Linux o fornecem como pacote separado.

```
python3 --version
python3 -m tkinter
```

Resultado esperado: o primeiro comando mostra a versão; o segundo abre uma pequena janela de demonstração de Tk. Feche essa janela para continuar. A versão utilizada nesta validação foi Python 3.12.14 com Tcl/Tk 9.0.4.

Sistema	Como começar
Windows	Use o instalador oficial do Python com Tcl/Tk. No terminal, py -3 -m tkinter e py -3 batalha_naval_gui.py.

Sistema	Como começar
Linux Debian/Ubuntu	Se o Tkinter estiver ausente no Python do sistema: sudo apt install python3-tk. Abra o programa em uma sessão de desktop.
macOS	Use uma distribuição Python com Tk compatível, como o instalador oficial. Confira com python3 -m tkinter antes de abrir o jogo.

Se o comando python3 não existir, tente python ou, no Windows, py -3. Se aparecer No module named tkinter, o Tk não está disponível nesse interpretador. Se aparecer no display name ou couldn’t connect to display, falta uma sessão gráfica. Instalar pacotes de pip não substitui um desktop ou uma tela virtual.

2.1. Abra a nova versão

Extraia Projeto_Batalha_Naval_V2.zip. Abra um terminal dentro da pasta Batalha-Naval-Python que contém os dois arquivos .py. Execute:

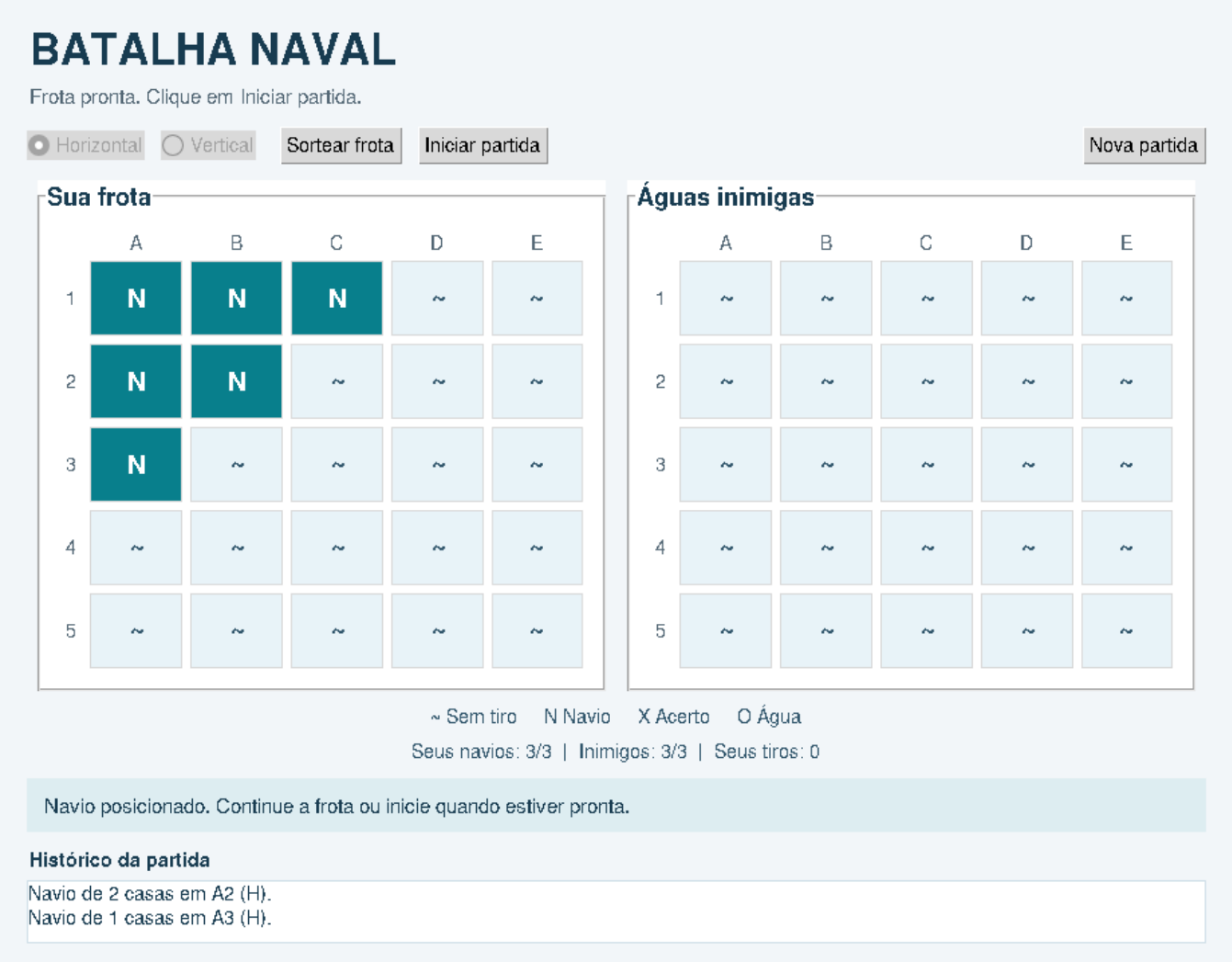
```
python3 batalha_naval_gui.py
```

A janela abre com a frota humana vazia e o adversário escondido. A orientação inicial é horizontal. O próximo tamanho aparece acima dos controles. Iniciar fica bloqueado até os três navios estarem prontos.

2.2. Prepare a frota por cliques

1. Com Horizontal selecionado, clique em A1 na grade Sua frota: o navio de 3 casas ocupa A1/B1/C1.
2. Clique em A2: o navio de 2 casas ocupa A2/B2.
3. Clique em A3: o navio de 1 casa ocupa A3.
4. Clique em Iniciar partida. A frota adversária continua oculta.

Você também pode selecionar Vertical antes de cada navio. O clique marca a casa inicial; o navio cresce para a direita em H ou para baixo em V. Uma tentativa fora do limite ou que cruza outro navio é rejeitada sem consumir o tamanho pendente. Sortear frota substitui a preparação manual por uma configuração válida.



Preparação real dos testes: navios de 3, 2 e 1 casas colocados manualmente. Iniciar está habilitado.

2.3. Ataque e acompanhe os turnos

Clique em uma casa da grade Águas inimigas. O símbolo muda para X quando há dano ou O quando o tiro atinge água. O computador responde após uma breve pausa de 350 ms; durante a pausa, os alvos ficam bloqueados. Depois da resposta, apenas casas ainda não atacadas ficam disponíveis.

O histórico lista tiros e resultados. A mensagem destacada informa a vez e os erros. Os contadores mostram os navios que ainda não afundaram; um navio parcialmente atingido continua sendo contado. Ao vencer ou perder, os ataques são bloqueados e a frota adversária restante é revelada. Nova partida volta à preparação.

Símbolo	Interpretação
~	Casa sem tiro; o conteúdo inimigo ainda é desconhecido.
N	Parte intacta revelada: frota própria ou frota inimiga após o fim.
X	Parte atingida. O histórico distingue acertou de afundou.

Símbolo	Interpretação
O	Tiro na água; é a letra O, não o número zero.

3. Construir as regras: batalha_naval.py

Crie o arquivo batalha_naval.py na pasta de estudo. Acrescente os blocos abaixo na ordem. Todos os métodos de uma classe mantêm quatro espaços antes de def; seu corpo tem oito. Quando começar outra classe ou uma função fora dela, volte à margem esquerda.

Passo 2 — Modele o navio

Objetivo: Definir tamanho, coordenadas ocupadas e danos. Uma casa da grade deve apontar ao objeto Navio, não a uma cópia.

Arquivo: batalha_naval.py • **linhas 1–17**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
"""Batalha Naval: regras do trabalho e jogo no terminal."""
import random
import sys


class Navio:
    def __init__(self, tamanho):
        if type(tamanho) is not int or tamanho < 1:
            raise ValueError("O tamanho do navio deve ser um inteiro positivo.")
        self.tamanho = tamanho
        self.posicoes = []
        self.acertos = []

    def esta_afundado(self):
        return len(self.acertos) == self.tamanho
```

Entenda cada linha

Linha 1 — Texto que documenta o arquivo. As aspas triplas criam uma docstring, consultável com help(). Não executa uma regra do jogo.

Linha 2 — Importa a biblioteca padrão de sorteios. Será usada para variar posições e tiros do computador; não exige instalação.

Linha 3 — Importa recursos do interpretador. sys.argv permitirá escolher a demonstração pela linha de comando.

Linha 4 — Em branco: separa os blocos; não executa instruções.

Linha 5 — Em branco: separa os blocos; não executa instruções.

Linha 6 — Define um tipo de objeto que representa um navio. Cada instância terá seu próprio tamanho, posições e danos.

Linha 7 — Inicializador chamado por Navio(3), por exemplo. self é a instância que está sendo criada; tamanho é o número de casas que ela ocupa.

Linha 8 — Só aceita inteiros positivos. O teste de tipo exato também rejeita True, pois bool é uma subclasse de int em Python.

Linha 9 — Interrompe a criação com uma mensagem clara. ValueError sinaliza que um valor recebido não respeita as regras.

Linha 10 — Guarda o tamanho na instância. Dois navios diferentes podem guardar tamanhos diferentes.

Linha 11 — Começa uma lista vazia das coordenadas ocupadas. Ela será preenchida apenas depois de um posicionamento válido.

Linha 12 — Guarda as coordenadas que já receberam dano. Começa vazia porque nenhum tiro atingiu o navio.

Linha 13 — Em branco: separa os blocos; não executa instruções.

Linha 14 — Método obrigatório que consulta o estado do navio sem modificar seus dados.

Linha 15 — Retorna True somente quando o número de partes atingidas é igual ao tamanho. Um navio novo não está afundado.

Linha 16 — Em branco: separa os blocos; não executa instruções.

Linha 17 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Navio; n = Navio(3); print(n.tamanho, n.acertos,
    n.esta_afundado())"
```

Resultado esperado: 3 [] False. O navio novo não está afundado.

Passo 3 — Crie o tabuleiro e valide coordenadas

Objetivo: Construir linhas independentes e rejeitar índices externos antes de acessar a grade.

Arquivo: batalha_naval.py • **linhas 18–32**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
class Tabuleiro:
    def __init__(self, tamanho=5):
        if type(tamanho) is not int or not 1 <= tamanho <= 26:
            raise ValueError("O tabuleiro deve ter entre 1 e 26 casas por lado.")
        self.tamanho = tamanho
        self.grade = [[None for _ in range(tamanho)] for _ in range(tamanho)]
        self.navios = []
        self.tiros = {}

    def validar_coordenada(self, linha, coluna):
        if type(linha) is not int or type(coluna) is not int:
            raise ValueError("Linha e coluna devem ser números inteiros.")
        if not (0 <= linha < self.tamanho and 0 <= coluna < self.tamanho):
            raise ValueError("Coordenada fora do tabuleiro.")
```

Entenda cada linha

Linha 18 — Reúne o espaço de jogo, a frota e os tiros recebidos. É responsável por validar e modificar esses dados.

Linha 19 — Cria um tabuleiro quadrado. O parâmetro padrão 5 permite escrever apenas Tabuleiro().

Linha 20 — Restringe o tamanho a inteiros de 1 a 26. O limite 26 permite usar uma letra de A a Z por coluna.

Linha 21 — Rejeita tamanhos incompatíveis com a grade e com o cabeçalho por letras.

Linha 22 — Guarda o tamanho para validar coordenadas e construir a exibição.

Linha 23 — Cria uma lista de linhas independentes, cada uma com tamanho casas. None representa água. Evita [[None] * tamanho] * tamanho, que compartilharia a mesma linha.

Linha 24 — Armazena os objetos Navio posicionados neste tabuleiro, usados para verificar a derrota.

Linha 25 — Cria um dicionário de tiros recebidos: a chave será (linha, coluna), e o valor será o resultado do tiro.

Linha 26 — Em branco: separa os blocos; não executa instruções.

Linha 27 — Centraliza a validação usada tanto no posicionamento quanto no ataque. Retorna normalmente se a coordenada for válida.

Linha 28 — Impede índices decimais, textos e booleanos. Verifica o tipo antes de comparar números.

Linha 29 — Informa que a coordenada recebida tem o tipo errado.

Linha 30 — Verifica os dois limites. Uma grade 5×5 usa índices de 0 a 4; índices negativos são rejeitados explicitamente.

Linha 31 — Impede que um tiro ou navio acesse uma casa externa, inclusive o índice -1, que Python normalmente aceitaria.

Linha 32 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Tabuleiro; t = Tabuleiro(); t.validar_coordenada(4,
    4); print(t.tamanho, t.grade[0])"
```

Resultado esperado: 5 e uma lista com cinco None. Tente validar_coordenada(5, 0); ela deve gerar ValueError.

Passo 4 — Implemente o posicionamento completo

Objetivo: Calcular todas as partes, validar e só então confirmar. A validação integral impede um navio parcialmente colocado.

Arquivo: batalha_naval.py • **linhas 33–51**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def posicionar_navio(self, navio, linha, coluna, orientacao):
        if navio.posicoes:
            raise ValueError("Este navio já foi posicionado.")
        if orientacao not in ("H", "V"):
            raise ValueError("Use H para horizontal ou V para vertical.")
        self.validar_coordenada(linha, coluna)
        posicoes = []
        for deslocamento in range(navio.tamanho):
            nova_linha = linha + (deslocamento if orientacao == "V" else 0)
            nova_coluna = coluna + (deslocamento if orientacao == "H" else 0)
            self.validar_coordenada(nova_linha, nova_coluna)
            if self.grade[nova_linha][nova_coluna] is not None:
                raise ValueError("Um navio não pode sobrepor outro.")
            posicoes.append((nova_linha, nova_coluna))
        for nova_linha, nova_coluna in posicoes:
            self.grade[nova_linha][nova_coluna] = navio
        navio.posicoes = posicoes
```

```
        self.navios.append(navio)
```

Entenda cada linha

Linha 33 — Método obrigatório. Recebe o objeto Navio, a casa inicial e H (horizontal) ou V (vertical).

Linha 34 — Uma lista não vazia indica que esse objeto já foi colocado em algum tabuleiro.

Linha 35 — Evita que a mesma instância represente dois navios ou tenha suas posições sobrescritas.

Linha 36 — Aceita apenas as duas direções suportadas. Navios diagonais não fazem parte desta implementação.

Linha 37 — Explica o formato correto sem mudar o tabuleiro.

Linha 38 — Verifica o início antes de calcular as outras casas.

Linha 39 — Lista temporária que permite validar o navio inteiro antes de alterar a grade.

Linha 40 — Percorre 0, 1, ... até tamanho - 1. Cada deslocamento corresponde a uma parte do navio.

Linha 41 — Em V, avança pelas linhas. Em H, soma zero e mantém a linha. A expressão condicional escolhe uma das duas opções.

Linha 42 — Em H, avança pelas colunas. Em V, mantém a coluna. Juntas, estas fórmulas geram um navio reto.

Linha 43 — Valida cada parte, não apenas a primeira. Um navio que começa dentro, mas termina fora, é rejeitado.

Linha 44 — Uma casa que contém um objeto Navio já está ocupada. Esta verificação bloqueia tanto sobreposição quanto cruzamento.

Linha 45 — Interrompe a tentativa ainda na fase de validação. Nenhuma casa da tentativa foi escrita até aqui.

Linha 46 — Adiciona uma tupla de dois números à lista temporária. Tuplas representam um par fixo de coordenadas.

Linha 47 — Depois que todas as partes passaram pelas verificações, percorre as coordenadas aprovadas. Atribui os dois elementos de cada tupla a variáveis.

Linha 48 — Cada casa aponta para o mesmo objeto Navio. Assim, atingir qualquer parte modifica os danos do navio inteiro.

Linha 49 — Registra no navio a lista completa de casas que ele ocupa.

Linha 50 — Inclui o navio na frota apenas uma vez e somente depois de um posicionamento válido.

Linha 51 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Navio, Tabuleiro; t=Tabuleiro(); n=Navio(2);
    t.posicionar_navio(n,1,1,'H'); print(n.posicoes)"
```

Resultado esperado: [(1, 1), (1, 2)]. Um segundo navio cruzando essas casas deve ser rejeitado sem modificar a grade.

Passo 5 — Processe os tiros e o afundamento

Objetivo: Receber coordenadas, impedir repetição e devolver os três resultados obrigatórios.

Arquivo: batalha_naval.py • **linhas 52–65**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def receber_tiro(self, linha, coluna):
        self.validar_coordenada(linha, coluna)
        coordenada = (linha, coluna)
        if coordenada in self.tiros:
            raise ValueError("Esta coordenada já recebeu um tiro.")
        navio = self.grade[linha][coluna]
        if navio is None:
            resultado = "água"
        else:
            navio.acertos.append(coordenada)
            resultado = "afundou" if navio.esta_afundado() else "acertou"
        self.tiros[coordenada] = resultado
        return resultado
```

Entenda cada linha

Linha 52 — Método obrigatório que processa um tiro recebido. Retorna uma das três palavras exigidas no trabalho.

Linha 53 — Rejeita coordenadas externas ou de tipo inválido antes de acessar a grade.

Linha 54 — Monta uma tupla que pode ser usada como chave de dicionário; uma lista não poderia ser essa chave.

Linha 55 — Consulta as chaves do histórico para saber se essa casa já recebeu um tiro.

Linha 56 — Impede dano duplicado. A interface pedirá outro tiro sem dar a vez ao computador.

Linha 57 — Busca a casa atingida: ela contém None ou uma referência a um Navio.

Linha 58 — Uma casa vazia significa que o ataque atingiu água.

Linha 59 — Define o resultado sem alterar os danos de qualquer navio.

Linha 60 — Trata o caso em que a casa contém um navio.

Linha 61 — Registra uma parte atingida. A verificação de repetição anterior garante um dano por casa.

Linha 62 — Consulta o navio depois do dano: o último acerto retorna afundou; um acerto parcial retorna acertou.

Linha 63 — Guarda o resultado no tabuleiro atacado, tanto para água quanto para acerto e afundamento.

Linha 64 — Entrega o resultado ao método que fez a chamada. Não imprime aqui para separar as regras da interface.

Linha 65 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Navio, Tabuleiro; t=Tabuleiro();
```

```
t.posicionar_navio(Navio(2),1,1,'H'); print(t.receber_tiro(1,1), t.receber_tiro(1,2),
t.receber_tiro(4,4))"
```

Resultado esperado: acertou afundou água. O segundo tiro destrói o navio de duas casas.

Passo 6 — Faça a exibição textual

Objetivo: Converter o estado em um desenho sem alterar os dados. A versão gráfica não depende desse desenho, mas ele facilita o estudo e a demonstração.

Arquivo: batalha_naval.py • **linhas 66–85**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
def exibir(self, revelar_navios=False):
    letras = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[:self.tamanho]
    linhas = ["    " + "  ".join(letras)]
    for linha in range(self.tamanho):
        simbolos = []
        for coluna in range(self.tamanho):
            resultado = self.tiros.get((linha, coluna))
            if resultado == "água":
                simbolo = "O"
            elif resultado in ("acertou", "afundou"):
                simbolo = "X"
            elif revelar_navios and self.grade[linha][coluna] is not None:
                simbolo = "N"
            else:
                simbolo = "~"
            simbolos.append(simbolo)
        linhas.append(f"{linha + 1:>2}  " + "  ".join(simbolos))
    return "\n".join(linhas)
```

Entenda cada linha

Linha 66 — Método adicional que produz uma string com o tabuleiro. Por padrão, esconde partes ainda não atingidas.

Linha 67 — Recorta o alfabeto para obter apenas as letras necessárias; tamanho 5 produz ABCDE.

Linha 68 — Cria a primeira linha com o cabeçalho. join intercala dois espaços entre as letras.

Linha 69 — Percorre as linhas internas, de zero até tamanho - 1.

Linha 70 — Lista temporária dos símbolos da linha atual.

Linha 71 — Percorre as casas da linha, da esquerda para a direita.

Linha 72 — get consulta o dicionário e retorna None quando ainda não houve tiro; evita um erro por chave ausente.

Linha 73 — Detecta uma casa em que um tiro anterior errou a frota.

Linha 74 — O é a letra maiúscula, não o número zero. Representa um tiro na água.

Linha 75 — Trata juntos os dois resultados que atingiram um navio.

Linha 76 — Mostra X em cada parte atingida, mesmo quando o navio inteiro já afundou.

Linha 77 — Só revela um navio não atingido se a opção for verdadeira e a casa estiver ocupada.

Linha 78 — N permite enxergar a própria frota. Nunca será exibido no tabuleiro inimigo durante a partida.

Linha 79 — Trata todas as casas restantes, ainda desconhecidas no modo oculto.

Linha 80 — O til significa casa sem tiro; no modo oculto, ela pode conter água ou um navio.

Linha 81 — Acumula o símbolo da casa para montar a linha completa.

Linha 82 — Converte o índice zero em número de linha visível. :>2 alinha esse número à direita em duas posições.

Linha 83 — Une as linhas com quebras de linha e devolve o desenho pronto, sem imprimir dentro da classe.

Linha 84 — Em branco: separa os blocos; não executa instruções.

Linha 85 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Tabuleiro; print(Tabuleiro().exibir())"
```

Resultado esperado: Cabeçalho A–E, números 1–5 e 25 casas com ~.

Passo 7 — Implemente o jogador e a derrota

Objetivo: Separar tiros realizados de tiros recebidos, e verificar todos os navios da própria frota.

Arquivo: batalha_naval.py • **linhas 86–102**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
class Jogador:
    def __init__(self, nome, tamanho=5):
        self.nome = nome
        self.tabuleiro = Tabuleiro(tamanho)
        self.registro_ataques = {}

    def atacar(self, inimigo, linha, coluna):
        if inimigo is self:
            raise ValueError("O jogador não pode atacar a si mesmo.")
        resultado = inimigo.tabuleiro.receber_tiro(linha, coluna)
        self.registro_ataques[(linha, coluna)] = resultado
        return resultado

    def verificar_derrota(self):
        return bool(self.tabuleiro.navios) and all(n.esta_afundado() for n in
    self.tabuleiro.navios)
```

Entenda cada linha

Linha 86 — Representa o participante, com seu próprio tabuleiro e histórico dos ataques que realizou.

Linha 87 — Recebe o nome e, opcionalmente, o tamanho de seu tabuleiro.

Linha 88 — Guarda o nome para as mensagens da partida.

Linha 89 — Cria um tabuleiro exclusivo deste jogador. O adversário receberá outra instância.

Linha 90 — Guarda os tiros feitos por este jogador. É diferente de tabuleiro.tiros, que contém os tiros recebidos.

Linha 91 — Em branco: separa os blocos; não executa instruções.

Linha 92 — Método obrigatório. Recebe o jogador inimigo e a coordenada interna do tiro.

Linha 93 — Compara a identidade dos objetos. Evita que um participante ataque o próprio tabuleiro por engano.

Linha 94 — Rejeita o ataque antes de alterar qualquer histórico.

Linha 95 — Delega as regras ao tabuleiro do adversário. Se houver um erro, a execução para antes de registrar o tiro.

Linha 96 — Após um tiro válido, registra a coordenada e seu resultado no atacante, cumprindo o enunciado.

Linha 97 — Permite que a interface mostre a resposta ao jogador.

Linha 98 — Em branco: separa os blocos; não executa instruções.

Linha 99 — Método obrigatório que consulta apenas a frota do próprio jogador.

Linha 100 — A frota precisa existir e todos os navios devem estar afundados. bool evita que all([]), que é True, declare derrota antes do jogo.

Linha 101 — Em branco: separa os blocos; não executa instruções.

Linha 102 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Jogador, Navio; a=Jogador('A'); b=Jogador('B');
    b.tabuleiro.posicionar_navio(Navio(1),0,0,'H'); print(a.atacar(b,0,0),
    b.verificar_derrota(), a.registro_ataques)"
```

Resultado esperado: afundou True e {(0, 0): "afundou"}. B tinha um único navio.

Passo 8 — Converta coordenadas e sorteie a frota

Objetivo: Transformar A1 em índices e testar candidatos finitos de colocação. shuffle evita repetir o mesmo candidato; choice será usado nos tiros.

Arquivo: batalha_naval.py • **linhas 103–134**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
def ler_coordenada(texto, tamanho=5):
    texto = texto.strip().upper()
    letras = "ABCDEFGHIJKLMNOPQRSTUVWXYZ"[:tamanho]
    if len(texto) < 2 or texto[0] not in letras or not texto[1:].isdigit():
        raise ValueError("Entrada inválida. Use uma letra e um número, como A1.")
    coluna = letras.index(texto[0])
    linha = int(texto[1:]) - 1
    if not 0 <= linha < tamanho:
        raise ValueError("Coordenada fora do tabuleiro.")
    return linha, coluna


def posicionar_frota(jogador):
    tabuleiro = jogador.tabuleiro
```

```
    candidatos = []
    for linha in range(tabuleiro.tamanho):
        for coluna in range(tabuleiro.tamanho):
            for orientacao in ("H", "V"):
                candidatos.append((linha, coluna, orientacao))
    for tamanho in (3, 2, 1):
        navio = Navio(tamanho)
        random.shuffle(candidatos)
        for linha, coluna, orientacao in candidatos:
            try:
                tabuleiro.posicionar_navio(navio, linha, coluna, orientacao)
            except ValueError:
                continue
            break
        else:
            raise RuntimeError("Não foi possível posicionar a frota neste tabuleiro.")
```

Entenda cada linha

Linha 103 — Função que traduz a entrada humana A1 em índices internos (0, 0). Não lê input diretamente; assim pode ser testada com textos conhecidos.

Linha 104 — Remove espaços apenas nas extremidades e converte letras para maiúsculas. Dessa forma, a1 e A1 são equivalentes.

Linha 105 — Obtém as letras válidas para o tamanho informado.

Linha 106 — Exige pelo menos uma letra e um algarismo. O operador or interrompe a avaliação quando encontra uma condição verdadeira, protegendo texto[0] em uma entrada vazia.

Linha 107 — Rejeita formatos como 1A, A-1 ou A 1 com uma orientação para corrigir a entrada.

Linha 108 — Busca a posição da letra na string. A está na posição zero; B, na posição um.

Linha 109 — Converte o trecho numérico em inteiro e subtrai um: a linha visível 1 corresponde ao índice 0.

Linha 110 — Verifica o intervalo depois da conversão; A0 e A6 são inválidos em um tabuleiro 5×5.

Linha 111 — Retorna um erro em vez de uma coordenada que não existe.

Linha 112 — Devolve uma tupla com os dois índices. O chamador pode separá-los usando linha, coluna = ler_coordenada(...).

Linha 113 — Em branco: separa os blocos; não executa instruções.

Linha 114 — Em branco: separa os blocos; não executa instruções.

Linha 115 — Função adicional para colocar automaticamente a frota padrão do participante. Foi escolhida para simplificar a partida no terminal.

Linha 116 — Cria um nome local para o mesmo objeto Tabuleiro, sem fazer uma cópia.

Linha 117 — Lista de todas as combinações de casa inicial e orientação que serão tentadas.

Linha 118 — Percorre todas as linhas possíveis.

Linha 119 — Para cada linha, percorre todas as colunas possíveis.

Linha 120 — Para cada casa, considera as duas direções permitidas.

Linha 121 — Armazena uma tentativa como uma tupla de três valores. Em 5×5, haverá 50 candidatos.

Linha 122 — Escolhe uma frota de três navios, com seis casas no total. Estes tamanhos são uma decisão adicional, não uma exigência do PDF.

Linha 123 — Cria um objeto novo para cada tamanho; não reutiliza um navio já posicionado.

Linha 124 — Embaralha a lista existente. Faz tentativas em ordem aleatória, sem repetir o mesmo candidato para um navio.

Linha 125 — Tenta cada posição possível, separando os três elementos da tupla em variáveis.

Linha 126 — Inicia um bloco em que um posicionamento inválido pode gerar ValueError.

Linha 127 — Usa o método obrigatório como única autoridade das regras. A função de sorteio não escreve diretamente na grade.

Linha 128 — Trata tentativas que saem do tabuleiro ou sobrepõem um navio já colocado.

Linha 129 — Passa ao próximo candidato do laço interno. A validação anterior manteve o estado intacto.

Linha 130 — Quando uma tentativa funciona, interrompe o laço de candidatos e passa ao próximo navio da frota.

Linha 131 — Este else pertence ao for: executa apenas se todos os candidatos terminarem sem break, ou seja, nenhum serviu.

Linha 132 — Falha de forma explícita, em vez de ficar sorteando para sempre. A frota padrão pressupõe o tabuleiro 5×5 usado por jogar().

Linha 133 — Em branco: separa os blocos; não executa instruções.

Linha 134 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -c "from batalha_naval import Jogador, ler_coordenada, posicionar_frota;
    j=Jogador('Teste'); posicionar_frota(j); print(ler_coordenada('D3'), [n.tamanho for n in
     j.tabuleiro.navios])"
```

Resultado esperado: (2, 3) [3, 2, 1]. O sorteio muda as posições; os tamanhos permanecem os mesmos.

Passo 9 — Complete a versão de terminal e a demonstração

Objetivo: Manter uma forma simples de experimentar as regras sem janela. O bloco __main__ impede iniciar input() quando o módulo for importado pela interface.

Arquivo: batalha_naval.py • **linhas 135–200**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
def demonstrar():
    ana = Jogador("Ana")
    computador = Jogador("Computador")
    ana.tabuleiro.posicionar_navio(Navio(1), 4, 0, "H")
    computador.tabuleiro.posicionar_navio(Navio(2), 1, 1, "H")
    computador.tabuleiro.posicionar_navio(Navio(1), 4, 4, "H")
    for rodada, texto in enumerate(("B2", "C2", "D3", "B5", "E5")):
        linha, coluna = ler_coordenada(texto)
        resultado = ana.atacar(computador, linha, coluna)
```

```
        print(f"Ana atirou em {texto}: {resultado}")
        print(computador.tabuleiro.exibir())
        if computador.verificar_derrota():
            print("Ana venceu!")
            break
        resultado = computador.atacar(ana, 0, rodada)
        print(f"Computador atirou em {chr(65 + rodada)}1: {resultado}")


def jogar():
    print("BATALHA NAVAL - Você contra o computador")
    nome = input("Seu nome: ").strip() or "Jogador"
    humano = Jogador(nome)
    computador = Jogador("Computador")
    posicionar_frota(humano)
    posicionar_frota(computador)
    alvos = [(l, c) for l in range(5) for c in range(5)]
    print("Frota: navios de 3, 2 e 1 casas. Digite A1 a E5, ou sair.")
    print("Legenda: ~ sem tiro | N seu navio | X acerto | O tiro na água")
    while True:
        print("\nSeu tabuleiro:")
        print(humano.tabuleiro.exibir(revelar_navios=True))
        print("\nSeus ataques ao inimigo:")
        print(computador.tabuleiro.exibir())
        texto = input("Seu tiro: ").strip()
        if texto.lower() == "sair":
            print("Partida encerrada.")
            return
        try:
            linha, coluna = ler_coordenada(texto)
            resultado = humano.atacar(computador, linha, coluna)
        except ValueError as erro:
            print(erro)
            continue
        print(f"Você: {resultado}")
        if computador.verificar_derrota():
            print(computador.tabuleiro.exibir())
            print(f"{humano.nome} venceu!")
            return
        linha, coluna = random.choice(alvos)
        alvos.remove((linha, coluna))
        resultado = computador.atacar(humano, linha, coluna)
        print(f"Computador atirou em {chr(65 + coluna)}{linha + 1}: {resultado}")
        if humano.verificar_derrota():
            print(humano.tabuleiro.exibir(revelar_navios=True))
            print("Computador venceu!")
            return


if __name__ == "__main__":
    try:
        if "--demo" in sys.argv:
            demonstrar()
        else:
            jogar()
    except (EOFError, KeyboardInterrupt):
        print("\nPartida encerrada.")
```

Entenda cada linha

Linha 135 — Cria uma demonstração reproduzível dos resultados água, acertou e afundou, seguida de uma vitória.

Linha 136 — Cria o primeiro jogador, com seu próprio tabuleiro e registro de ataques.

Linha 137 — Cria o adversário da demonstração.

Linha 138 — Coloca um navio de uma casa em A5 no tabuleiro de Ana; os tiros do computador serão na primeira linha e não o atingirão.

Linha 139 — Coloca um navio de duas casas em B2 e C2, que permite mostrar dano parcial e afundamento.

Linha 140 — Coloca outro navio em E5; afundar o primeiro ainda não significa vencer.

Linha 141 — Percorre uma sequência fixa de ataques. enumerate fornece índices de rodada começando em zero junto com as coordenadas.

Linha 142 — Usa a mesma conversão de coordenadas da partida interativa.

Linha 143 — Executa o ataque pelas classes obrigatórias, atualizando o defensor e o registro de Ana.

Linha 144 — Exibe uma f-string: os campos entre chaves são substituídos pelos valores atuais.

Linha 145 — Mostra os tiros já feitos, mantendo as partes inimigas ainda intactas escondidas.

Linha 146 — Testa o fim da partida imediatamente após o ataque de Ana.

Linha 147 — Só anuncia vitória quando os dois navios adversários estão afundados.

Linha 148 — Encerra a sequência; o computador não ganha uma jogada depois de perder.

Linha 149 — Alterna a vez: o computador atira em A1, B1, C1 e D1, todos na água nesta demonstração.

Linha 150 — chr converte um código numérico em letra. 65 corresponde a A; a rodada escolhe a coluna exibida.

Linha 151 — Em branco: separa os blocos; não executa instruções.

Linha 152 — Em branco: separa os blocos; não executa instruções.

Linha 153 — Conduz uma partida no terminal. Coordena as classes e a entrada/saída; as regras continuam nos métodos obrigatórios.

Linha 154 — Mostra a identificação do jogo.

Linha 155 — Lê o nome, remove espaços externos e usa Jogador quando a resposta estiver vazia. or devolve o segundo valor se o primeiro for vazio.

Linha 156 — Cria o participante humano com o tabuleiro padrão 5×5.

Linha 157 — Cria um segundo participante com estado independente.

Linha 158 — Sorteia e posiciona os navios do humano, respeitando limites e sobreposição.

Linha 159 — Faz o mesmo para a frota adversária. Os sorteios podem resultar em configurações diferentes.

Linha 160 — Cria as 25 coordenadas que o computador ainda pode atacar. A compreensão equivale a dois laços e um append.

Linha 161 — Explica as escolhas da partida e o formato de entrada. O navio de tamanho 3 só afunda com três acertos distintos.

Linha 162 — Explica os quatro símbolos. No tabuleiro inimigo, ~ significa informação desconhecida.

Linha 163 — Repete os turnos até uma vitória ou saída voluntária; os comandos return encerram esta função.

Linha 164 — Inicia um bloco visual, separado por uma quebra de linha.

Linha 165 — Mostra a própria frota, incluindo N nas partes que ainda não foram atingidas.

Linha 166 — Identifica o mapa dos tiros já realizados contra o computador.

Linha 167 — Mostra o adversário no modo oculto. Não expõe os navios ainda não atingidos.

Linha 168 — Lê a coordenada escolhida pelo humano.

Linha 169 — Permite encerrar sem completar a partida, aceitando também SAIR e Sair.

Linha 170 — Confirma a saída voluntária.

Linha 171 — Encerra jogar() e não executa o turno do computador.

Linha 172 — Protege a conversão e o ataque contra entradas inválidas e tiros repetidos.

Linha 173 — Traduz a casa da interface para índices internos de lista.

Linha 174 — Faz um tiro válido contra o adversário e salva o resultado no humano.

Linha 175 — Captura o erro esperado e dá o nome erro ao objeto que contém a mensagem.

Linha 176 — Exibe a explicação específica: formato errado, coordenada externa ou tiro repetido.

Linha 177 — Volta ao início do while sem executar o ataque do computador. A tentativa inválida não consome a vez.

Linha 178 — Exibe a resposta água, acertou ou afundou ao ataque recém-feito.

Linha 179 — Consulta a frota adversária antes de permitir sua próxima ação.

Linha 180 — Mostra o resultado final dos ataques, incluindo o tiro que encerrou a partida.

Linha 181 — Informa que o humano afundou todos os navios do computador.

Linha 182 — Encerra imediatamente; impede uma jogada extra depois da derrota adversária.

Linha 183 — Escolhe uma coordenada ainda disponível. O computador não usa a grade inimiga para decidir: seus tiros são aleatórios.

Linha 184 — Retira o alvo escolhido para garantir que o computador não o use de novo.

Linha 185 — Executa o turno adversário usando exatamente as mesmas regras e registros do humano.

Linha 186 — Converte os índices internos para uma coordenada como C4 e informa o resultado.

Linha 187 — Verifica se este tiro afundou o último navio do humano.

Linha 188 — Mostra o estado final da frota humana e todos os tiros recebidos.

Linha 189 — Informa a vitória adversária.

Linha 190 — Finaliza a partida antes de pedir um novo tiro.

Linha 191 — Em branco: separa os blocos; não executa instruções.

Linha 192 — Em branco: separa os blocos; não executa instruções.

Linha 193 — Executa a interface apenas quando este arquivo é iniciado diretamente. Ao importá-lo nos testes, as classes ficam disponíveis sem iniciar input().

Linha 194 — Permite tratar uma interrupção do terminal de maneira amigável.

Linha 195 — Verifica se a opção de demonstração está nos argumentos. sys.argv[0] é o nome do arquivo; os demais itens são opções fornecidas pelo usuário.

Linha 196 — Executa o exemplo fixo, que não precisa de teclado nem sorteios.

Linha 197 — Seleciona o fluxo normal quando a opção --demo não foi informada.

Linha 198 — Inicia a partida interativa contra o computador.

Linha 199 — Trata o fim inesperado da entrada e Ctrl+C. Não captura todos os erros, evitando esconder defeitos de programação.

Linha 200 — Exibe uma mensagem curta em vez de um traceback quando o usuário interrompe a partida.

Confira antes de continuar

```
python3 batalha_naval.py --demo
```

Resultado esperado: A demonstração termina com Ana venceu!. O desenho depois de B5 coincide com o exemplo do enunciado. Este bloco preserva a versão anterior de terminal.

4. Construir a janela: batalha_naval_gui.py

Crie batalha_naval_gui.py na mesma pasta do módulo de regras. A janela importa Jogador, Navio e posicionar_frota; não copia as classes. Acrescente todos os blocos abaixo na ordem. Durante a construção, alguns métodos citam outros definidos em passos posteriores: isso é permitido, desde que a classe esteja completa antes de ser instanciada.

Antes do último passo, confira a sintaxe com python3 -m py_compile batalha_naval_gui.py. Isso não prova que a interface funciona; apenas verifica que o arquivo pode ser compilado. Abrir a janela e executar testes reais será a etapa seguinte.

4.1. Entenda a programação por eventos

No terminal, input() para a função até alguém digitar. Em Tkinter, mainloop() recebe os eventos; um botão chama o método associado a command. A função passada não leva parênteses, pois queremos executá-la no clique, não ao criar o widget. after(350, função) agenda uma chamada para depois sem bloquear a janela.

Fase	Ações permitidas
posicionamento	Colocar navios, escolher H/V e sortear; iniciar só com frota completa.
jogando	Atacar uma casa inimiga ainda não usada.
computador	Aguardar resposta; alvos bloqueados, reinício disponível.
encerrado	Consultar o resultado e começar outra partida; sem novos tiros.

Estas fases formam uma máquina de estados simples. A tela usa a fase para bloquear controles; os próprios callbacks verificam a fase novamente. A combinação protege tanto o clique comum quanto uma chamada direta durante os testes.

Passo 10 — Importe as regras e defina as cores

Objetivo: Preparar a biblioteca gráfica, os quatro símbolos e a conversão de índices para nomes de casas. As cores acompanham letras, não substituem a informação.

Arquivo: batalha_naval_gui.py • **linhas 1–19**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
"""Versão gráfica de Batalha Naval, usando as regras do trabalho."""
import random
import tkinter as tk
from tkinter import ttk
from batalha_naval import Jogador, Navio, posicionar_frota


CORES = {
    "~": ("#E8F3F8", "#31566C"),
    "N": ("#087F8C", "#FFFFFF"),
```

```
    "X": ("#B83C4B", "#FFFFFF"),
    "O": ("#DCE4EA", "#334B5B"),
}


def nome_casa(linha, coluna):
    return f"{chr(65 + coluna)}{linha + 1}"
```

Entenda cada linha

Linha 1 — Documenta a finalidade do arquivo. Esta versão apresenta botões e mensagens, mas usa as mesmas regras da versão de terminal.

Linha 2 — Importa os sorteios da biblioteca padrão; o computador escolherá entre casas ainda não usadas.

Linha 3 — Importa Tkinter com o nome curto tk. Ele cria a janela, os botões, os textos e o laço de eventos.

Linha 4 — Importa os controles temáticos, usados nos botões de ação e na escolha de orientação.

Linha 5 — Reutiliza a implementação do trabalho. A interface não precisa copiar as regras de limites, danos ou derrota.

Linha 6 — Em branco: separa os blocos; não executa instruções.

Linha 7 — Em branco: separa os blocos; não executa instruções.

Linha 8 — Cria um dicionário que liga cada símbolo a um par: cor de fundo e cor do texto. A legenda também usa letras, então não depende apenas das cores.

Linha 9 — Casa sem tiro: fundo azul claro e texto escuro.

Linha 10 — Navio revelado: fundo verde-azulado e letra branca.

Linha 11 — Parte atingida: fundo vermelho e X branco.

Linha 12 — Tiro na água: fundo cinza e letra O escura.

Linha 13 — Fecha o dicionário de cores. Uma vírgula após o último item é aceita por Python.

Linha 14 — Em branco: separa os blocos; não executa instruções.

Linha 15 — Em branco: separa os blocos; não executa instruções.

Linha 16 — Função pequena para converter índices internos em uma coordenada visível.

Linha 17 — chr(65) é A; a coluna desloca a letra. Somar um à linha transforma o índice zero no número 1.

Linha 18 — Em branco: separa os blocos; não executa instruções.

Linha 19 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: O arquivo contém imports, CORES e nome_casa. Ainda não há uma janela aberta.

Passo 11 — Defina a classe da janela e suas variáveis

Objetivo: Guardar janela, atraso, tarefa pendente e textos vinculados aos controles. StringVar comunica mudanças de texto aos rótulos.

Arquivo: batalha_naval_gui.py • **linhas 20–32**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
class JanelaBatalhaNaval:
    def __init__(self, raiz, atraso_computador=350):
        self.raiz = raiz
        self.atraso_computador = atraso_computador
        self.tarefa = None
        self.status = tk.StringVar(master=raiz)
        self.instrucao = tk.StringVar(master=raiz)
        self.contagem = tk.StringVar(master=raiz)
        self.orientacao = tk.StringVar(master=raiz, value="H")
        self.montar_interface()
        self.nova_partida()
        raiz.protocol("WM_DELETE_WINDOW", self.fechar)
```

Entenda cada linha

Linha 20 — Organiza os controles e a partida gráfica. É uma classe adicional: as três classes obrigatórias continuam no módulo de regras.

Linha 21 — Recebe a janela raiz e o atraso da resposta, em milissegundos. Os testes usam zero para não depender de espera real.

Linha 22 — Guarda a referência à janela para montar controles, agendar eventos e encerrá-la.

Linha 23 — Guarda o intervalo entre o tiro humano e a resposta adversária.

Linha 24 — Guarda o identificador de uma resposta agendada; None significa que não há uma resposta pendente.

Linha 25 — Texto associado à mensagem principal. Alterar seu valor atualiza automaticamente o rótulo correspondente.

Linha 26 — Texto associado à orientação do que fazer na fase atual.

Linha 27 — Texto associado aos contadores de navios e tiros.

Linha 28 — Valor compartilhado pelos dois controles de orientação. A opção inicial é horizontal.

Linha 29 — Cria os widgets uma única vez. As partidas futuras reutilizam a mesma janela e os mesmos botões.

Linha 30 — Inicializa os jogadores, a frota inimiga, o histórico e o estado da primeira partida.

Linha 31 — Faz o botão de fechar a janela chamar fechar(), que cancela respostas pendentes antes de destruir a janela.

Linha 32 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: A classe pode referenciar métodos posteriores, mas não tente instanciá-la até completar os próximos passos.

Passo 12 — Monte a estrutura da interface

Objetivo: Criar título, orientação, ações, tabuleiros, legenda, contadores, mensagem e histórico. pack organiza blocos; grid organiza casas dentro de outro contêiner.

Arquivo: batalha_naval_gui.py • **linhas 33–67**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def montar_interface(self):
        self.raiz.title("Batalha Naval - Versão gráfica")
        self.raiz.geometry("920x720")
        self.raiz.minsize(820, 680)
        self.raiz.configure(bg="#F2F6F9")
        painel = tk.Frame(self.raiz, bg="#F2F6F9", padx=20, pady=16)
        painel.pack(fill="both", expand=True)
        tk.Label(painel, text="BATALHA NAVAL", font=("Arial", 23, "bold"), bg="#F2F6F9",
    fg="#173B50").pack(anchor="w")
        tk.Label(painel, textvariable=self.instrucao, font=("Arial", 11), bg="#F2F6F9",
    fg="#526674").pack(anchor="w", pady=(3, 12))
        barra = tk.Frame(painel, bg="#F2F6F9")
        barra.pack(fill="x", pady=(0, 12))
        self.radios = []
        for valor, titulo in (("H", "Horizontal"), ("V", "Vertical")):
            radio = ttk.Radiobutton(barra, text=titulo, variable=self.orientacao,
    value=valor)
            radio.pack(side="left", padx=(0, 12))
            self.radios.append(radio)
        self.botao_sortear = ttk.Button(barra, text="Sortear frota",
    command=self.sortear_frota)
        self.botao_sortear.pack(side="left", padx=6)
        self.botao_iniciar = ttk.Button(barra, text="Iniciar partida", command=self.iniciar)
        self.botao_iniciar.pack(side="left", padx=6)
        self.botao_nova = ttk.Button(barra, text="Nova partida", command=self.nova_partida)
        self.botao_nova.pack(side="right")
        tabuleiros = tk.Frame(painel, bg="#F2F6F9")
        tabuleiros.pack(fill="both", expand=True)
        tabuleiros.columnconfigure(0, weight=1)
        tabuleiros.columnconfigure(1, weight=1)
        self.botoes_proprios = self.criar_grade(tabuleiros, "Sua frota", 0, True)
        self.botoes_inimigos = self.criar_grade(tabuleiros, "Águas inimigas", 1, False)
        tk.Label(painel, text="~ Sem tiro     N Navio     X Acerto     O Água",
    bg="#F2F6F9", fg="#31566C", font=("Arial", 10)).pack(pady=(8, 5))
        tk.Label(painel, textvariable=self.contagem, bg="#F2F6F9", fg="#31566C",
    font=("Arial", 10)).pack()
        tk.Label(painel, textvariable=self.status, wraplength=780, anchor="w",
    justify="left", bg="#DDEDF1", fg="#173B50", font=("Arial", 11), padx=12,
    pady=10).pack(fill="x", pady=(10, 10))
        tk.Label(painel, text="Histórico da partida", bg="#F2F6F9", fg="#173B50",
    font=("Arial", 10, "bold")).pack(anchor="w")
        self.historico = tk.Listbox(painel, height=5, font=("Arial", 10), bg="#FFFFFF",
    fg="#173B50", borderwidth=0, highlightthickness=1, highlightbackground="#CCDDE6")
        self.historico.pack(fill="x", pady=(5, 0))
```

Entenda cada linha

Linha 33 — Constrói a apresentação. Este método não coloca navios nem aplica danos.

Linha 34 — Define o texto da barra de título do sistema operacional.

Linha 35 — Escolhe o tamanho inicial da janela em pixels; o usuário ainda pode redimensionar.

Linha 36 — Evita uma janela tão pequena que os tabuleiros e controles deixem de caber.

Linha 37 — Define o fundo claro da janela.

Linha 38 — Cria um contêiner com margens internas para organizar toda a interface.

Linha 39 — Faz o painel preencher a área disponível e crescer com a janela.

Linha 40 — Mostra o título maior, alinhado à esquerda. A tupla define fonte, tamanho e negrito.

Linha 41 — Mostra instruções que acompanham a fase. textvariable liga o rótulo a uma StringVar.

Linha 42 — Cria uma área para orientação e botões de ação.

Linha 43 — Mantém a barra horizontal e acrescenta espaço abaixo.

Linha 44 — Guarda os controles de orientação para habilitá-los apenas durante a preparação.

Linha 45 — Cria as duas opções que compartilham a variável orientacao.

Linha 46 — O controle escreve H ou V em orientacao quando é selecionado.

Linha 47 — Coloca as duas opções na mesma linha, com espaçamento.

Linha 48 — Guarda cada controle para atualizar seu estado depois.

Linha 49 — Cria o botão que substitui o posicionamento manual por uma frota válida sorteada.

Linha 50 — Posiciona o botão de sorteio na barra.

Linha 51 — Associa o clique ao método iniciar. Passa a referência do método sem parênteses, para não executá-lo durante a criação.

Linha 52 — Coloca o botão de início ao lado do sorteio.

Linha 53 — Cria um controle sempre disponível para começar novamente.

Linha 54 — Alinha o reinício à direita da barra.

Linha 55 — Cria o contêiner que receberá os dois tabuleiros lado a lado.

Linha 56 — Permite que essa região ocupe o espaço principal da janela.

Linha 57 — Distribui parte do espaço horizontal adicional à primeira coluna.

Linha 58 — Dá o mesmo peso à segunda coluna, mantendo larguras semelhantes.

Linha 59 — Cria a grade do humano e guarda os botões em um dicionário por coordenada.

Linha 60 — Cria a grade adversária. Os cliques nela serão usados para atacar.

Linha 61 — Exibe uma legenda textual para interpretar as casas, inclusive sem distinguir as cores.

Linha 62 — Mostra quantos navios ainda estão inteiros ou parcialmente intactos e quantos tiros você realizou.

Linha 63 — Cria a mensagem destacada. wraplength quebra textos compridos para caber na janela.

Linha 64 — Identifica a lista de acontecimentos abaixo.

Linha 65 — Cria uma lista consultável de posicionamentos, tiros e resultado final; o usuário não edita os itens.

Linha 66 — Usa toda a largura do painel para o histórico.

Linha 67 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: Cada widget tem seu pai. Não misture pack e grid para widgets que compartilham o mesmo pai; o código usa pais diferentes.

Passo 13 — Crie as grades e associe os cliques

Objetivo: Guardar 25 botões por tabuleiro e conectar cada casa à coordenada correta. Os parâmetros padrão da lambda congelam os valores do laço.

Arquivo: batalha_naval_gui.py • **linhas 68–84**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def criar_grade(self, pai, titulo, coluna_painel, propria):
        quadro = tk.LabelFrame(pai, text=titulo, font=("Arial", 13, "bold"), bg="#FFFFFF",
    fg="#173B50", padx=12, pady=12)
        quadro.grid(row=0, column=coluna_painel, padx=8, sticky="nsew")
        botoes = {}
        comando = self.posicionar if propria else self.atirar
        for indice in range(5):
            tk.Label(quadro, text=chr(65 + indice), bg="#FFFFFF", fg="#526674").grid(row=0,
    column=indice + 1)
            tk.Label(quadro, text=str(indice + 1), bg="#FFFFFF",
    fg="#526674").grid(row=indice + 1, column=0, padx=5)
            quadro.columnconfigure(indice + 1, weight=1)
        for linha in range(5):
            for coluna in range(5):
                botao = tk.Button(quadro, width=4, height=2, font=("Arial", 14, "bold"),
    relief="flat", borderwidth=0, takefocus=True)
                botao.configure(command=lambda l=linha, c=coluna: comando(l, c))
                botao.grid(row=linha + 1, column=coluna + 1, padx=3, pady=3, sticky="ew")
                botoes[(linha, coluna)] = botao
        return botoes
```

Entenda cada linha

Linha 68 — Constrói um tabuleiro com rótulos e 25 botões. propria escolhe o comportamento dos cliques.

Linha 69 — Agrupa as casas dentro de uma moldura com título.

Linha 70 — Posiciona o tabuleiro na coluna escolhida. sticky permite esticar o quadro nas direções disponíveis.

Linha 71 — Dicionário que permitirá consultar diretamente o botão de uma casa, como botoes[(1, 2)].

Linha 72 — Usa o clique da frota para colocar navios e o clique das águas inimigas para disparar tiros.

Linha 73 — Percorre os cinco rótulos de linhas e colunas.

Linha 74 — Adiciona as letras A a E acima da grade. A coluna zero fica reservada aos números.

Linha 75 — Adiciona os números 1 a 5 à esquerda das casas.

Linha 76 — Permite distribuir o espaço entre as cinco colunas de casas.

Linha 77 — Percorre cada linha do tabuleiro.

Linha 78 — Para cada linha, cria as cinco casas.

Linha 79 — Cria uma casa clicável, com letra legível e possibilidade de foco pelo teclado.

Linha 80 — Guarda os valores atuais nos parâmetros padrão da lambda. Sem isso, todos os cliques usariam os valores finais do laço.

Linha 81 — Organiza a casa abaixo dos rótulos de coordenadas. Os pequenos espaços separam visualmente os botões.

Linha 82 — Guarda a referência ao widget pela sua coordenada interna.

Linha 83 — Devolve os 25 widgets para o método que montou a interface.

Linha 84 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: botoes[(linha, coluna)] permite atualizar uma casa. Sem l=linha e c=coluna, todos os cliques poderiam usar a última coordenada.

Passo 14 — Escreva o histórico

Objetivo: Centralizar os acontecimentos e mostrar a entrada mais recente sem o usuário ter de rolar a cada turno.

Arquivo: batalha_naval_gui.py • **linhas 85–88**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def registrar(self, texto):
        self.historico.insert(tk.END, texto)
        self.historico.see(tk.END)
```

Entenda cada linha

Linha 85 — Centraliza a escrita no histórico visual.

Linha 86 — Acrescenta o acontecimento ao final da lista.

Linha 87 — Rola a lista para que o último acontecimento fique visível.

Linha 88 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: Os registros visuais são adicionais. O registro obrigatório de ataques continua no objeto Jogador.

Passo 15 — Prepare o cancelamento de respostas

Objetivo: Guardar e cancelar o identificador de after antes de reiniciar ou destruir widgets.

Arquivo: batalha_naval_gui.py • **linhas 89–93**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def cancelar_tarefa(self):
        if self.tarefa is not None:
            self.raiz.after_cancel(self.tarefa)
            self.tarefa = None
```

Entenda cada linha

Linha 89 — Remove uma resposta adversária que tenha sido agendada, mas ainda não executada.

Linha 90 — Só tenta cancelar quando há um identificador salvo.

Linha 91 — Informa ao laço de eventos que esse callback não deve mais ocorrer.

Linha 92 — Limpa a referência para não tentar cancelar a mesma tarefa novamente.

Linha 93 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: cancelar_tarefa não altera frota. Só remove uma resposta agendada, se existir.

Passo 16 — Inicialize uma nova partida

Objetivo: Criar jogadores novos, sortear a frota adversária, limpar os dados humanos e voltar à preparação.

Arquivo: batalha_naval_gui.py • **linhas 94–107**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def nova_partida(self):
        self.cancelar_tarefa()
        self.humano = Jogador("Você")
        self.computador = Jogador("Computador")
        posicionar_frota(self.computador)
        self.pendentes = [3, 2, 1]
        self.alvos = [(l, c) for l in range(5) for c in range(5)]
        self.fase = "posicionamento"
        self.orientacao.set("H")
        self.historico.delete(0, tk.END)
        self.status.set("Escolha H/V e clique na sua frota, ou use Sortear frota.")
        self.registrar("Nova partida: posicione navios de 3, 2 e 1 casas.")
        self.atualizar()
```

Entenda cada linha

Linha 94 — Recomeça com jogadores e históricos novos. Também funciona enquanto o computador está aguardando para responder.

Linha 95 — Evita que uma resposta da partida antiga atinja a frota da partida nova.

Linha 96 — Cria o participante com frota ainda vazia e sem dano.

Linha 97 — Cria um adversário sem registros de partidas anteriores.

Linha 98 — Sorteia a frota inimiga usando a mesma função de regras da versão de terminal.

Linha 99 — Define a ordem dos navios que o humano deve posicionar manualmente.

Linha 100 — Reinicializa as 25 casas disponíveis para os tiros do computador.

Linha 101 — Coloca a partida na preparação, bloqueando ataques.

Linha 102 — Restaura a orientação horizontal para a primeira colocação.

Linha 103 — Remove os acontecimentos da partida anterior.

Linha 104 — Apresenta as duas formas de preparar os navios.

Linha 105 — Registra o começo da preparação no histórico.

Linha 106 — Redesenha as casas e ajusta quais controles estão habilitados.

Linha 107 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: Na abertura e em Nova partida, pendentes volta a [3, 2, 1] e a fase a posicionamento.

Passo 17 — Posicione um navio pelo clique

Objetivo: Usar o tamanho pendente e H/V para chamar o método obrigatório. Só consumir o tamanho após sucesso.

Arquivo: batalha_naval_gui.py • **linhas 108–121**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def posicionar(self, linha, coluna):
        if self.fase != "posicionamento" or not self.pendentes:
            return
        tamanho = self.pendentes[0]
        try:
            self.humano.tabuleiro.posicionar_navio(Navio(tamanho), linha, coluna,
    self.orientacao.get())
        except ValueError as erro:
            self.status.set(str(erro))
            return
        self.pendentes.pop(0)
        self.registrar(f"Navio de {tamanho} casas em {nome_casa(linha, coluna)}
    ({self.orientacao.get()}).")
        self.status.set("Navio posicionado. Continue a frota ou inicie quando estiver
    pronta.")
        self.atualizar()
```

Entenda cada linha

Linha 108 — Trata um clique na própria grade durante a preparação.

Linha 109 — Ignora cliques fora da preparação ou depois de completar a frota.

Linha 110 — Sai sem modificar os jogadores.

Linha 111 — Seleciona o próximo tamanho que ainda não foi colocado.

Linha 112 — Protege a tentativa contra erros esperados de limite e sobreposição.

Linha 113 — Cria um navio novo e delega todas as validações ao tabuleiro, usando H/V escolhido na tela.

Linha 114 — Recebe a mensagem gerada pelas regras de posicionamento.

Linha 115 — Mostra o motivo da rejeição diretamente na janela, sem uma caixa modal que interrompa o uso.

Linha 116 — Não remove nenhum tamanho pendente; o humano pode tentar outra casa.

Linha 117 — Retira o tamanho da fila somente depois de uma colocação válida.

Linha 118 — Anota a casa inicial e a orientação usadas.

Linha 119 — Confirma a colocação sem confundir preparação com ataque.

Linha 120 — Mostra as partes com N e libera Iniciar quando não há mais tamanhos pendentes.

Linha 121 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: Erro de limite ou cruzamento aparece na mensagem. A fila de tamanhos não muda quando a tentativa falha.

Passo 18 — Ofereça o sorteio da própria frota

Objetivo: Substituir a preparação parcial por uma frota inteira válida, sem manter navios anteriores misturados ao sorteio.

Arquivo: batalha_naval_gui.py • **linhas 122–131**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def sortear_frota(self):
        if self.fase != "posicionamento":
            return
        self.humano = Jogador("Você")
        posicionar_frota(self.humano)
        self.pendentes = []
        self.status.set("Frota sorteada. Clique em Iniciar partida.")
        self.registrar("Sua frota foi substituída por um novo sorteio.")
        self.atualizar()
```

Entenda cada linha

Linha 122 — Permite trocar a preparação manual por uma frota automática, inclusive se alguns navios já foram colocados.

Linha 123 — Não permite alterar uma frota depois de começar o combate.

Linha 124 — Ignora chamadas fora da preparação, mesmo que feitas diretamente.

Linha 125 — Substitui o participante por um novo, removendo navios parcialmente colocados antes do sorteio.

Linha 126 — Preenche a frota inteira de tamanhos 3, 2 e 1 por posições válidas.

Linha 127 — Indica que todos os navios estão prontos; não há mais tamanhos a posicionar.

Linha 128 — Explica que o combate ainda depende de uma ação explícita de início.

Linha 129 — Mantém o histórico legível quando o usuário refaz a preparação.

Linha 130 — Redesenha a frota e atualiza os botões.

Linha 131 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: Depois do sorteio, pendentes está vazia e o botão de início pode ser liberado. Após começar o combate, o método não altera a frota.

Passo 19 — Libere o início apenas quando a frota estiver pronta

Objetivo: Exigir preparação completa e mudar para a vez humana. Não confiar apenas na aparência do botão.

Arquivo: batalha_naval_gui.py • **linhas 132–139**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def iniciar(self):
        if self.fase != "posicionamento" or self.pendentes:
            return
        self.fase = "jogando"
        self.status.set("Sua vez: escolha uma casa nas águas inimigas.")
        self.registrar("Combate iniciado. Você joga primeiro.")
        self.atualizar()
```

Entenda cada linha

Linha 132 — Muda da preparação para o primeiro turno humano.

Linha 133 — Exige estar na preparação e ter colocado os três navios. O método verifica a condição, além de o botão ficar bloqueado.

Linha 134 — Impede iniciar com uma frota vazia ou incompleta.

Linha 135 — Define que o próximo clique nas águas inimigas será um tiro humano.

Linha 136 — Orienta onde o próximo clique deve acontecer.

Linha 137 — Separa visualmente a preparação dos turnos de ataque.

Linha 138 — Bloqueia controles de posicionamento e libera alvos ainda não usados.

Linha 139 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: iniciar retorna sem mudar a fase quando faltam navios. Com frota completa, passa a jogando.

Passo 20 — Ataque por clique e agende a resposta

Objetivo: Aplicar o tiro, verificar vitória imediatamente e, se necessário, bloquear os alvos antes do turno adversário.

Arquivo: batalha_naval_gui.py • **linhas 140–156**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def atirar(self, linha, coluna):
        if self.fase != "jogando":
            return
        try:
            resultado = self.humano.atacar(self.computador, linha, coluna)
        except ValueError as erro:
            self.status.set(str(erro))
            return
        self.registrar(f"Você → {nome_casa(linha, coluna)}: {resultado}.")
        if self.computador.verificar_derrota():
            self.finalizar("Você venceu! Toda a frota inimiga afundou.")
            return
        self.fase = "computador"
        self.status.set(f"Você: {resultado}. O computador está escolhendo uma casa...")
        self.atualizar()
        self.tarefa = self.raiz.after(self.atraso_computador, self.responder_computador)
```

Entenda cada linha

Linha 140 — Processa um clique nas águas inimigas no turno humano.

Linha 141 — Rejeita ações na preparação, durante a espera do computador ou depois do fim.

Linha 142 — Impede um clique extra de executar outro ataque no mesmo turno.

Linha 143 — Protege a chamada das regras contra uma coordenada inválida ou repetida.

Linha 144 — Usa o método obrigatório, que aplica o tiro no inimigo e registra o resultado no humano.

Linha 145 — Trata uma tentativa rejeitada pela lógica do tabuleiro.

Linha 146 — Mostra o erro sem agendar a resposta adversária.

Linha 147 — Preserva a vez do humano e os históricos anteriores.

Linha 148 — Registra o tiro válido no histórico da janela.

Linha 149 — Verifica a frota adversária imediatamente após o dano.

Linha 150 — Anuncia o resultado e bloqueia a grade no mesmo turno.

Linha 151 — Não agenda uma resposta para um adversário já derrotado.

Linha 152 — Fecha os alvos enquanto a resposta adversária está pendente; isso evita cliques duplos.

Linha 153 — Mostra o resultado humano e explica a breve espera.

Linha 154 — Pinta a casa atacada e desabilita os outros alvos antes de agendar o próximo turno.

Linha 155 — Agenda a resposta sem travar a janela. after devolve o identificador que poderá ser cancelado no reinício ou fechamento.

Linha 156 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: O último acerto finaliza sem agendar resposta. Um clique duplo não entra em um segundo turno humano.

Passo 21 — Execute a resposta do computador

Objetivo: Escolher somente um alvo disponível, aplicar as mesmas regras, verificar derrota e devolver a vez.

Arquivo: batalha_naval_gui.py • **linhas 157–171**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def responder_computador(self):
        self.tarefa = None
        if self.fase != "computador":
            return
        linha, coluna = random.choice(self.alvos)
        self.alvos.remove((linha, coluna))
        resultado = self.computador.atacar(self.humano, linha, coluna)
        self.registrar(f"Computador → {nome_casa(linha, coluna)}: {resultado}.")
        if self.humano.verificar_derrota():
            self.finalizar("Computador venceu! Sua frota foi afundada.")
            return
        self.fase = "jogando"
        self.status.set(f"Computador: {resultado}. Agora é sua vez.")
        self.atualizar()
```

Entenda cada linha

Linha 157 — Callback executado pelo laço de eventos depois do atraso configurado.

Linha 158 — A tarefa já começou; seu identificador não representa mais uma resposta pendente.

Linha 159 — Proteção adicional contra uma resposta que não corresponda ao turno atual.

Linha 160 — Sai sem modificar as frotas se a fase mudou.

Linha 161 — Sorteia entre os alvos restantes, sem examinar os navios do humano para escolher.

Linha 162 — Remove a casa escolhida e impede que o computador a use novamente.

Linha 163 — Aplica o ataque adversário pelas mesmas regras utilizadas pelo humano.

Linha 164 — Mostra o tiro e o resultado no histórico visual.

Linha 165 — Consulta se esse ataque destruiu o último navio humano.

Linha 166 — Encerra a partida, pinta o estado final e bloqueia novos ataques.

Linha 167 — Evita devolver uma vez ao humano depois da derrota.

Linha 168 — Devolve o controle ao participante humano quando ele ainda tem frota viva.

Linha 169 — Informa o resultado adversário e qual participante deve agir.

Linha 170 — Atualiza o próprio tabuleiro atingido e libera apenas alvos inimigos ainda não atacados.

Linha 171 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: O alvo é removido após a escolha. O computador não consulta a grade humana para decidir onde atirar.

Passo 22 — Centralize o encerramento

Objetivo: Marcar a fase final e mostrar o resultado em dois locais da tela.

Arquivo: batalha_naval_gui.py • **linhas 172–177**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def finalizar(self, mensagem):
        self.fase = "encerrado"
        self.status.set(mensagem)
        self.registrar(mensagem)
        self.atualizar()
```

Entenda cada linha

Linha 172 — Centraliza o encerramento tanto na vitória humana quanto na vitória do computador.

Linha 173 — Marca a partida como terminada para bloquear callbacks de ataque.

Linha 174 — Destaca o vencedor e o motivo do fim.

Linha 175 — Guarda o resultado também no histórico.

Linha 176 — Bloqueia os tabuleiros e revela a frota inimiga restante para conferir o resultado final.

Linha 177 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: Finalizar não permite novos ataques. A atualização revela a frota adversária restante apenas depois do fim.

Passo 23 — Sincronize símbolos, controles e contadores

Objetivo: Ler os objetos para redesenhar a janela e decidir onde cliques são permitidos. A tela não deve virar uma segunda fonte de regras.

Arquivo: batalha_naval_gui.py • **linhas 178–211**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def atualizar(self):
```

```
    for propria, jogador, botoes in ((True, self.humano, self.botoes_proprios), (False,
self.computador, self.botoes_inimigos)):
        tabuleiro = jogador.tabuleiro
        for (linha, coluna), botao in botoes.items():
            resultado = tabuleiro.tiros.get((linha, coluna))
            if resultado == "água":
                simbolo = "O"
            elif resultado in ("acertou", "afundou"):
                simbolo = "X"
            elif (propria or self.fase == "encerrado") and
tabuleiro.grade[linha][coluna] is not None:
                simbolo = "N"
            else:
                simbolo = "~"
            habilitado = propria and self.fase == "posicionamento" and
bool(self.pendentes)
            habilitado = habilitado or (not propria and self.fase == "jogando" and
resultado is None)
            fundo, texto = CORES[simbolo]
            botao.configure(text=simbolo, bg=fundo, fg=texto, disabledforeground=texto,
activebackground=fundo, state="normal" if habilitado else "disabled")
    preparando = self.fase == "posicionamento"
    self.botao_sortear.configure(state="normal" if preparando else "disabled")
    self.botao_iniciar.configure(state="normal" if preparando and not self.pendentes
else "disabled")
    for radio in self.radios:
        radio.configure(state="normal" if preparando and self.pendentes else "disabled")
    if preparando and self.pendentes:
        self.instrucao.set(f"Posicione um navio de {self.pendentes[0]} casas na sua
frota.")
    elif preparando:
        self.instrucao.set("Frota pronta. Clique em Iniciar partida.")
    elif self.fase == "encerrado":
        self.instrucao.set("Partida concluída. Use Nova partida para jogar novamente.")
    else:
        self.instrucao.set("Ataque clicando nas águas inimigas. Sua frota fica à
esquerda.")
    vivos_h = sum(not n.esta_afundado() for n in self.humano.tabuleiro.navios)
    vivos_c = sum(not n.esta_afundado() for n in self.computador.tabuleiro.navios)
    self.contagem.set(f"Seus navios: {vivos_h}/{len(self.humano.tabuleiro.navios)}  |
Inimigos: {vivos_c}/3  |  Seus tiros: {len(self.humano.registro_ataques)}")
```

Entenda cada linha

Linha 178 — Sincroniza a tela com o estado das classes. As cores e letras são uma apresentação; não substituem o estado real.

Linha 179 — Percorre os dois tabuleiros com seus widgets, identificando qual pertence ao humano.

Linha 180 — Usa a referência ao tabuleiro da iteração atual.

Linha 181 — Percorre cada coordenada e o botão que a apresenta.

Linha 182 — Consulta os tiros recebidos nessa casa; None indica que ainda não houve tiro.

Linha 183 — Uma casa atingida sem navio recebe o símbolo de erro.

Linha 184 — Usa a letra O para representar água atingida.

Linha 185 — Um navio atingido recebe o mesmo símbolo para dano parcial ou completo.

Linha 186 — Marca a parte acertada; o resultado específico permanece no histórico.

Linha 187 — Revela navios próprios desde a preparação; navios adversários só são revelados depois do fim da partida.

Linha 188 — Indica uma parte intacta que pode ser revelada neste momento.

Linha 189 — Trata uma casa sem tiro cujo navio não pode ser mostrado, ou uma casa vazia.

Linha 190 — Mantém o conteúdo desconhecido nas águas inimigas durante o combate.

Linha 191 — Libera cliques na própria grade apenas enquanto ainda há navios a colocar.

Linha 192 — Libera cliques nas águas inimigas apenas na vez humana e em casas sem tiro anterior.

Linha 193 — Recupera o par de cores correspondente à letra que será exibida.

Linha 194 — Atualiza texto, cores e interação. disabledforeground mantém a letra legível mesmo em botões bloqueados.

Linha 195 — Variável local que simplifica as condições dos controles de preparação.

Linha 196 — Sorteio só fica disponível antes do combate.

Linha 197 — Início só fica disponível quando os três navios já foram colocados.

Linha 198 — Percorre os dois seletores H/V.

Linha 199 — Orientação só pode mudar enquanto houver um navio a posicionar.

Linha 200 — Seleciona a mensagem de preparação ainda incompleta.

Linha 201 — Mostra qual tamanho será colocado no próximo clique.

Linha 202 — Trata uma frota completa que ainda não entrou no combate.

Linha 203 — Orienta a ação seguinte sem iniciar automaticamente.

Linha 204 — Trata o estado final, em que nenhum tiro é permitido.

Linha 205 — Explica como começar uma nova tentativa.

Linha 206 — Trata as fases de combate humano e resposta do computador.

Linha 207 — Mantém uma instrução geral; a mensagem principal informa a vez atual.

Linha 208 — Soma True como 1 e False como 0 para contar os navios humanos ainda não afundados.

Linha 209 — Conta os navios inimigos ainda não afundados sem expor suas posições.

Linha 210 — Atualiza os contadores, usando os objetos como fonte dos dados. A frota inimiga padrão sempre tem três navios.

Linha 211 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: N do adversário não aparece durante o combate; os controles acompanham a fase. Um navio parcialmente atingido ainda conta como vivo.

Passo 24 — Feche sem deixar callbacks pendentes

Objetivo: Cancelar a resposta antes de destruir a janela para que nenhum método tente alterar widgets removidos.

Arquivo: batalha_naval_gui.py • **linhas 212–216**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
    def fechar(self):
        self.cancelar_tarefa()
        self.raiz.destroy()
```

Entenda cada linha

Linha 212 — Trata o fechamento da janela com limpeza das tarefas pendentes.

Linha 213 — Impede callbacks que tentariam acessar widgets destruídos.

Linha 214 — Destrói a janela e encerra seu laço de eventos.

Linha 215 — Em branco: separa os blocos; não executa instruções.

Linha 216 — Em branco: separa os blocos; não executa instruções.

Confira antes de continuar

```
python3 -m py_compile batalha_naval_gui.py
```

Resultado esperado: O fechamento encerra o laço de eventos e não deixa uma resposta antiga executar.

Passo 25 — Abra a janela e inicie o laço de eventos

Objetivo: Criar Tk, apresentar erro claro se não houver uma sessão gráfica e chamar mainloop. A importação nos testes não abre automaticamente a interface.

Arquivo: batalha_naval_gui.py • **linhas 217–223**.
Faça: acrescente o bloco abaixo na ordem indicada. Preserve os espaços à esquerda e o nível de indentação.

```
if __name__ == "__main__":
    try:
        raiz = tk.Tk()
    except tk.TclError as erro:
        raise SystemExit("Não foi possível abrir a janela. Execute em uma sessão gráfica com
     Python e Tkinter instalados.") from erro
    JanelaBatalhaNaval(raiz)
    raiz.mainloop()
```

Entenda cada linha

Linha 217 — Só abre a janela quando este arquivo é executado diretamente. Importá-lo nos testes não cria uma interface automaticamente.

Linha 218 — Permite informar uma falha de inicialização gráfica de forma compreensível.

Linha 219 — Cria uma janela real. Precisa de Tk instalado e de uma sessão gráfica disponível.

Linha 220 — Captura a falha de Tk, como ausência de monitor ou tela virtual.

Linha 221 — Encerra com uma mensagem prática. Não inventa uma janela nem afirma que a interface abriu sem um display.

Linha 222 — Monta a interface e inicia a preparação da primeira partida.

Linha 223 — Mantém a janela ativa, processando cliques, atualizações e respostas agendadas até o usuário fechá-la.

Confira antes de continuar

```
python3 batalha_naval_gui.py
```

Resultado esperado: Agora a classe está completa. Execute o comando abaixo: a janela deve abrir na fase de posicionamento.

5. Testar de verdade: regras, widgets e eventos

Passo 26 — Execute a suíte e interprete o resultado

Copie test_batalha_naval.py e test_batalha_naval_gui.py do pacote para a pasta de estudo. Abra um terminal em uma sessão de desktop e execute:

```
python3 -m unittest -v
```

Resultado esperado da versão entregue: Ran 47 tests e OK, sem testes gráficos pulados. Há 27 testes das regras/terminal e 20 testes da janela. Se aparecer skipped por falta de display, os testes gráficos não rodaram; não considere isso uma validação da interface.

Os testes gráficos criam tk.Tk() de verdade, montam os widgets e acionam comandos de botões com invoke(). A resposta agendada é executada por raiz.update(). Os objetos do jogo não são substituídos por simuladores. Os testes usam atraso zero para remover a espera, mantendo os mesmos callbacks.

Casos verificados	O que precisa acontecer
Preparação e orientação	Grade própria aceita H/V; limites e sobreposição não consomem o navio.
Sorteio e início	Sorteio substitui a frota parcial; combate só começa com os três navios.
Clíques e resultado	Acerto pinta X, água pinta O e o registro acompanha os objetos.
Bloqueio de turnos	Clique duplo e repetido não produzem outra resposta nem dano extra.
Vitória e derrota	Partida completa chega ao fim; o último tiro não permite jogada adicional.
Reinício e fechamento	Resposta agendada é cancelada; novos jogadores não herdam dano ou tiros antigos.

5.1. Como escrever um teste pequeno

```
import unittest
from batalha_naval import Navio

class TestNavio(unittest.TestCase):
    def test_novo_nao_afundou(self):
        navio = Navio(2)
        self.assertFalse(navio.esta_afundado())

if __name__ == "__main__":
    unittest.main()
```

1. Importe unittest e a classe que será exercitada.
2. Crie uma classe que herda unittest.TestCase.
3. Nomeie o método com test_ para ele ser descoberto.
4. Construa um objeto real.
5. Use assertFalse para exigir o resultado esperado.

6. Rode o teste. Se alterar esta_afundado para retornar True sempre, este teste precisa falhar.

assertEqual compara resultados; assertIs compara referências; assertRaises exige um erro esperado. Primeiro prepare o cenário, depois faça a ação e por fim confira o resultado. Não enfraqueça a verificação só para obter OK.

5.2. Como a janela foi validada neste ambiente

Como o ambiente de execução não possui monitor, foi iniciado Xvfb em uma tela virtual Linux :99, sem escuta TCP. Os pacotes oficiais Debian foram obtidos usando índices assinados e verificações de integridade; os arquivos foram extraídos em /workspace/dev-tools, sem alterar os pacotes do sistema. Todas as 47 verificações executaram nessa tela. Um teste adicional manteve o atraso real de 350 ms e confirmou que a janela processa outro evento durante a espera.

```
DISPLAY=:99 python3 -m unittest -v
```

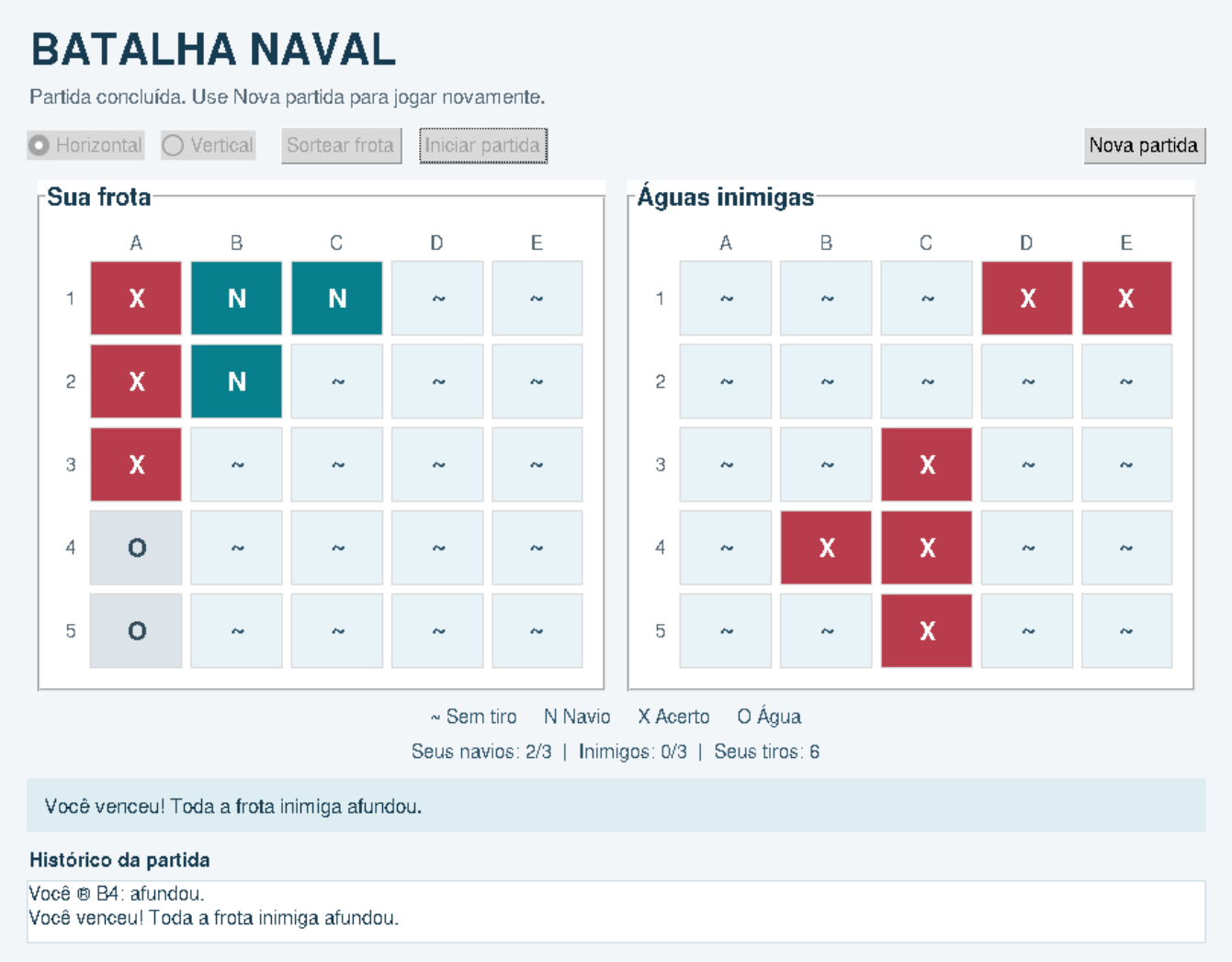
Esse comando só funciona quando a tela :99 já está em execução e há permissão de acesso. Em seu computador com desktop, use o comando normal sem DISPLAY. Em um Linux sem desktop com Xvfb/xauth instalados, a alternativa comum é:

```
xvfb-run -a python3 -m unittest -v
```

Além da suíte, docs/capturar_telas.py enviou eventos reais de botão pressionado/solto aos widgets: três posicionamentos, início, seis ataques humanos, cinco respostas adversárias, vitória e reinício. O último ataque não teve resposta. O script também registrou erros de callbacks; não houve nenhum.

O manual também foi verificado como receita de construção: seus 24 blocos de código foram acrescentados em arquivos vazios e compilados incrementalmente. Os dois arquivos reconstruídos ficaram idênticos aos entregues. Os sete comandos de conferência das regras foram executados e produziram os resultados descritos nos passos.

As imagens deste manual são capturas dessa janela, e não desenhos de como ela deveria aparecer. Para reproduzir as capturas, a ferramenta de documentação usa Pillow e uma sessão gráfica. Não é preciso Pillow para jogar ou executar a suíte de 47 testes.



Captura real de um cenário de teste concluído: seis partes atingidas, vitória e controles de ataque bloqueados.

A validação gráfica ocorreu em Linux com tela virtual; Windows e macOS recebem instruções de execução, mas não foram executados neste ambiente. Uma tela virtual verifica widgets e eventos, enquanto um teste manual no seu computador também confere fontes, escala e aparência do sistema.

6. Conferência final e dúvidas comuns

Passo 27 — Faça a conferência da entrega

1. Abra a janela e coloque um navio na borda que não comporte seu tamanho: a mensagem deve rejeitar a tentativa.
2. Complete a frota ou sorteie e inicie.
3. Faça um tiro e tente um segundo antes da resposta: ele deve ficar bloqueado.
4. Reinicie durante a espera: a frota nova não deve receber um tiro antigo.
5. Jogue até o fim; confira histórico, vencedor e bloqueio.
6. Execute a suíte e confira que nenhum teste gráfico foi pulado.

Se uma verificação falhar, leia a primeira mensagem e identifique a camada: regra, interface ou ambiente. Uma ValueError de coordenada pode ser o comportamento correto de uma tentativa inválida; um TclError por ausência de display é uma necessidade do ambiente. Não esconda

uma falha do programa com um except genérico.

Dúvida	Resposta prática
Por que não usar sleep(0.35)?	Ele bloqueia o mesmo fluxo que processa os eventos. after mantém a janela responsiva e permite cancelar a resposta.
Por que congelar l/c na lambda?	A função será executada depois. Sem parâmetros padrão, procuraria o valor das variáveis após o laço, quando ambas já chegaram ao fim.
Por que guardar o identificador da tarefa?	Para remover uma resposta pendente quando a partida muda ou a janela fecha.
Por que N não aparece no inimigo?	A atualização permite revelar o adversário apenas na fase encerrado, preservando o jogo durante o combate.
Por que registrar em dois lugares?	O atacante guarda os tiros realizados; o tabuleiro defensor guarda os recebidos. O histórico da janela é apenas uma apresentação adicional.
Por que a água não aplica dano?	A casa contém None. Só um objeto Navio recebe um novo acerto.
Por que frota vazia não perde?	all([]) é True; por isso verificar_derrota também exige bool(navios).
Posso mudar atributos diretamente?	Para estudar, você pode inspecioná-los. Para jogar, use os métodos: modificar grade, acertos ou históricos diretamente pode quebrar as regras.

6.1. Exercícios após concluir o manual

1. Antes de executar, preveja as casas de um navio vertical de 3 partes colocado em D2.
Resposta: D2/D3/D4; índices (1,3), (2,3), (3,3).
2. Explique por que um navio horizontal de 3 partes em D5 deve falhar.
3. Descreva as mudanças de fase de um turno completo.
4. Mude apenas as cores de CORES e confira que as regras e os testes continuam iguais.
5. Escreva um teste para a situação “não atacar a si mesmo” antes de alterar a regra.

A interface é básica e local, para um jogador contra o computador. Não inclui rede, salvamento de partidas ou adaptação automática para qualquer tamanho de tabuleiro. O módulo Tabuleiro permite outros tamanhos para estudo, mas os controles gráficos desta versão representam 5×5.

6.2. Arquivos do pacote

Arquivo ou pasta	Finalidade
batalha_naval.py	Regras do trabalho e terminal; 200 linhas explicadas.
batalha_naval_gui.py	Interface gráfica; 223 linhas explicadas.
test_batalha_naval*.py	47 testes das regras e da janela.
docs/Manual_Batalha_Naval_V2.pdf	Este manual de construção.
docs/manual_v2.md	Versão textual com blocos completos, adequada para copiar e pesquisar.
docs/imagens/	Capturas reais da janela usadas no manual.
docs/explicacoes*.json	Correspondência do código com as explicações.
docs/gerar_manual_v2.py e capturar_telas.py	Ferramentas opcionais de documentação; não necessárias para jogar.
docs/Guia_Batalha_Naval.pdf	Guia anterior da versão de terminal, mantido como referência.

Comece executando a versão pronta; depois reconstrua a parte de regras, confira os resultados e só então avance para a janela. Ao terminar, explique os cinco métodos obrigatórios e o caminho do clique até o dano sem consultar o manual.